# Mech Lab Tools

Documentazione degli strumenti di analisi

Giancarmine Sparso

30 marzo 2026

# Indice

**Introduzione** **2**

**1 io_utils.py** **3**
1.1 load_csv(path, ...) . . . 3

**2 stats_utils.py** **4**
2.1 median(x) . . . 4
2.2 weighted_mean(x, w=None) . . . 5
2.3 variance(x, w=None) . . . 6
2.4 covariance(x, y, w=None) . . . 7
2.5 standard_deviation(x, w=None) . . . 8

**3 plot_utils.py** **9**
3.1 histogram(x, bins, label, xlabel, ylabel, title, figsize, save_path, show_mean, show_std, dpi, ax) . . . 9
3.1.1 Parametri e figura . . . 10
3.1.2 Istogramma . . . 11
3.1.3 Elementi dell'istogramma . . . 11
3.1.4 Esempio d'uso . . . 13

**4 fit_utils.py** **14**
4.1 lin_fit(x, y, sigma_y, xlabel, ylabel, dpi, band, plot) . . . 14
4.1.1 Pesi e stima dei parametri . . . 15
4.1.2 Incertezza sui parametri . . . 15
4.1.3 Residui e sigma dei residui . . . 16
4.1.4 Grafico dati, retta e residui . . . 16

# Introduzione

*Mech Lab Tools* è una toolbox in Python che sto sviluppando per le esperienze di laboratorio del primo anno di fisica. L'obiettivo è fornire una base modulare e riutilizzabile per le operazioni ricorrenti nell'analisi dei dati sperimentali: statistica descrittiva e produzione di grafici.

Il progetto è organizzato in moduli distinti, ciascuno con un compito preciso, in modo che le stesse funzioni possano essere impiegate in esperienze diverse senza duplicazione di codice. I criteri guida dello sviluppo sono la trasparenza dei passaggi matematici, la riproducibilità dei risultati e la crescita graduale della toolbox man mano che le esigenze del corso aumentano.

Questa documentazione descrive ogni modulo in dettaglio — le funzioni implementate, le formule matematiche sottostanti, i parametri accettati e i valori restituiti — e costituisce un riferimento sia per l'utilizzo corretto degli strumenti sia per la verifica della correttezza delle implementazioni.

Le librerie esterne utilizzate nel progetto sono `NumPy`, `pandas` e `Matplotlib`. In tutto il codice vengono importate con i seguenti acronimi standard:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Gli snippet di codice sono formattati usando lo stile `gruvbox-light`.

Una riga ricorrente all'inizio di quasi tutte le funzioni è:

```
x = np.asarray(x, dtype=float)
```

Questa istruzione converte l'argomento `x` in un array NumPy di numeri in virgola mobile (`float64`), indipendentemente dal tipo con cui viene passato (lista, tupla, array intero, ecc.). Ciò garantisce che le operazioni vettoriali di NumPy funzionino correttamente e previene errori di tipo nelle divisioni o nelle operazioni matematiche. È una pratica difensiva comune nelle librerie numeriche in Python.

Capitolo 1

# io_utils.py

Il file *io_utils.py* raccoglie le funzioni di caricamento dati. Espone un'unica funzione, `load_csv`, che avvolge `pd.read_csv` aggiungendo validazione delle colonne e gestione dei valori mancanti.

## 1.1 load_csv(path, ...)

**Scopo**

Caricare un file CSV in un `DataFrame`, con supporto a separatori e decimali personalizzati, rinomina delle colonne e controllo dei valori mancanti.

**Parametri**

- **`path`** str o Path → percorso del file CSV
- **`sep`** str, default `","` → separatore di colonna
- **`decimal`** str, default `"."` → separatore decimale
- **`rename_columns`** dict o `None`, default `None` → mappa `{vecchio: nuovo}` per rinominare le colonne
- **`required_columns`** list o `None`, default `None` → lista di colonne obbligatorie
- **`drop_missing`** bool, default `False` → se `True` elimina le righe con NaN; altrimenti solleva eccezione

**Note**

Le righe che iniziano con # vengono ignorate; gli spazi dopo il separatore vengono rimossi automaticamente. Questi comportamenti sono fissi.

**Restituisce**

`pandas.DataFrame` con i dati del file.

**Eccezioni**

`ValueError` se una o più colonne di `required_columns` sono assenti; `ValueError` se il file contiene valori NaN e `drop_missing=False`.

### Implementazione

```python
def load_csv(path, sep=",", decimal=".", rename_columns=None,
             required_columns=None, drop_missing=False):
    df = pd.read_csv(Path(path), sep=sep, decimal=decimal,
                     comment='#', skipinitialspace=True)
    if rename_columns:
        df = df.rename(columns=rename_columns)
    if required_columns:
        missing = [col for col in required_columns if col not in df.columns]
        if missing:
            raise ValueError(f"Colonne mancanti: {missing}")
    if drop_missing:
        df = df.dropna()
    elif df.isna().any().any():
        raise ValueError("Il file contiene valori mancanti")
    return df
```

### Esempio d'uso

```python
# CSV con separatore ";" e decimale "," (formato italiano)
df = load_csv("misure.csv", sep=";", decimal=",",
              rename_columns={"t [s]": "t", "x [m]": "x"},
              required_columns=["t", "x"], drop_missing=True)
```

Capitolo 2

# stats_utils.py

Il file *stats_utils.py* raccoglie le funzioni statistiche di base utilizzate nell'analisi dei dati sperimentali. Le funzioni supportano pesi opzionali $w_i = 1/\sigma_i^2$, utili quando le misure hanno incertezze diverse.

Le funzioni disponibili sono:

- `median(x)`: calcola la mediana di un array di dati;
- `weighted_mean(x, w)`: calcola la media aritmetica, opzionalmente pesata;
- `variance(x, w)`: calcola la varianza tramite la formula $\mathrm{Var}(x) = E[x^2] - E[x]^2$;
- `covariance(x, y, w)`: calcola la covarianza tra due variabili tramite la formula $\mathrm{Cov}(x, y) = E[xy] - E[x]E[y]$;
- `standard_deviation(x, w)`: calcola la deviazione standard come $\sqrt{\mathrm{Var}(x)}$.

## 2.1 median(x)

**Scopo**

Calcolare la mediana di un array di dati numerici.

**Parametri**

`x` array-like (lista, tupla, array NumPy) → convertito in `float64`

**Vincoli**

Nessun vincolo aggiuntivo.

**Restituisce**

`float`: mediana di `x`.

**Eccezioni**

Nessuna eccezione esplicita; se `x` è vuoto, `np.median` restituisce `NaN` con un avviso di runtime.

### Implementazione

```
def median(x):
    x = np.asarray(x, dtype=float)
    return np.median(x)
```

La funzione converte l'input in un array `float64` e delega il calcolo a `np.median`.

### Significato matematico

Data una sequenza di $n$ osservazioni ordinate $x_{(1)} \leq x_{(2)} \leq \cdots \leq x_{(n)}$, la mediana è il valore centrale:

$$\mathrm{med}(x) = \begin{cases} x_{\left(\frac{n+1}{2}\right)} & \text{se } n \text{ è dispari} \\ \frac{x_{\left(\frac{n}{2}\right)} + x_{\left(\frac{n}{2}+1\right)}}{2} & \text{se } n \text{ è pari} \end{cases}$$

A differenza della media, la mediana è robusta rispetto ai valori anomali (outlier): un singolo dato molto distante dagli altri non la sposta significativamente.

### Esempio d'uso

```
x = [3.1, 2.7, 4.0, 2.9, 3.5]
print(median(x))  # 3.1
# Ordinati: [2.7, 2.9, 3.1, 3.5, 4.0] -> elemento centrale: 3.1
```

## 2.2 weighted_mean(x, w=None)

**Scopo**

Calcolare la media aritmetica di un array, con possibilità di pesare ogni dato in base alla sua incertezza.

**Parametri**

- `x` array-like → convertito in `float64`
- `w` array-like o `None` (default: `None`)

**Vincoli**

Se `w` ≠ `None`, i pesi devono essere pre-calcolati come $w_i = 1/\sigma_i^2$ e $\sum_i w_i > 0$. Se `w = None`, si usa $w_i = 1$ per tutti gli $i$.

**Restituisce**

`float`: media (pesata) di `x`.

**Eccezioni**

Nessuna eccezione esplicita; se $\sum_i w_i = 0$, NumPy esegue una divisione per zero floating-point e restituisce `NaN`.

---

### Implementazione

```
def weighted_mean(x, w=None):
    x = np.asarray(x, dtype=float)
    if w is None:
        w = np.ones(len(x))
    else:
        w = np.asarray(w, dtype=float)
    return np.sum(x * w) / np.sum(w)
```

Se non si passa un valore `w`, la funzione crea un array di tutti `1`, quindi ogni dato pesa uguale e si ottiene la media aritmetica classica.

### Significato matematico

Senza pesi ($w_i = 1$ per tutti) si ha la media aritmetica classica:

$$\bar{x} = \frac{1}{N} \sum_i x_i$$

Se vengono passati i pesi $w_i = 1/\sigma_i^2$, si calcola la media pesata:

$$\bar{x}_w = \frac{\sum_i w_i x_i}{\sum_i w_i}$$

dove i punti con incertezza minore (peso maggiore) contribuiscono di più al risultato.

### Esempio d'uso

```
import numpy as np
from stats_utils import weighted_mean

# Media semplice
x = [1.0, 2.0, 3.0, 4.0, 5.0]
print(weighted_mean(x))         # 3.0

# Media pesata: misure con incertezze diverse
x     = np.array([10.0, 10.5,  9.8])
sigma = np.array([ 0.2,  0.5,  0.1])
w     = 1 / sigma**2            # [25.0, 4.0, 100.0]
print(weighted_mean(x, w))      # ≈ 9.860 (dominato da x=9.8, sigma=0.1)
```

## 2.3 variance(x, w=None)

**Scopo**
Calcolare la varianza di un array, con possibilità di pesare ogni dato.

**Parametri**

- `x` array-like → convertito in `float64`
- `w` array-like o `None` (default: `None`)

**Vincoli**
Se `w` ≠ `None`, i pesi devono essere pre-calcolati come $w_i = 1/\sigma_i^2$ e $\sum_i w_i > 0$. Se `w = None`, si usa $w_i = 1$ per tutti gli $i$.

**Restituisce**
`float`: varianza di `x`.

**Eccezioni**
Nessuna eccezione esplicita; le stesse condizioni limite di `weighted_mean` si propagano (divisione per zero se $\sum_i w_i = 0$).

### Implementazione

```
def variance(x, w=None):
    x = np.asarray(x, dtype=float)
    return weighted_mean(x**2, w) - weighted_mean(x, w)**2
```

La funzione delega interamente il calcolo a `weighted_mean`, sfruttando l'identità algebrica tra media dei quadrati e quadrato della media.

### Significato matematico

La varianza è calcolata tramite l'identità:

$$\mathrm{Var}(x) = \overline{x^2} - \overline{x}^2$$

ovvero la media dei quadrati meno il quadrato della media. Con i pesi, la formula completa diventa:

$$\mathrm{Var}_w(x) = \frac{\sum_i w_i x_i^2}{\sum_i w_i} - \left(\frac{\sum_i w_i x_i}{\sum_i w_i}\right)^2$$

Senza pesi ($w_i = 1$ per tutti) si riduce alla forma classica:

$$\mathrm{Var}(x) = \frac{1}{N}\sum_i (x_i - \overline{x})^2$$

> **Nota:** tutte le funzioni di questo modulo usano statistiche descrittive della popolazione (divisore $N$), non stimatori campionari non distorti (divisore $N-1$). La notazione $E[x^2] - E[x]^2$ si riferisce pertanto alla media empirica del campione, non al valore atteso di una variabile aleatoria.

### Esempio d'uso

```
from stats_utils import variance

x = [1.0, 2.0, 3.0, 4.0, 5.0]
print(variance(x))   # 2.0
# media = 3, E[x^2] = (1+4+9+16+25)/5 = 11 -> 11 - 9 = 2
```

## 2.4 covariance(x, y, w=None)

**Scopo**

Calcolare la covarianza tra due array di dati, con possibilità di pesare ogni coppia.

**Parametri**

- `x` array-like → convertito in `float64`
- `y` array-like → convertito in `float64`
- `w` array-like o `None` (default: `None`)

**Vincoli**

`x` e `y` devono avere la stessa lunghezza. Se `w` ≠ `None`, i pesi devono essere pre-calcolati come $w_i = 1/\sigma_i^2$ e $\sum_i w_i > 0$.

**Restituisce**

`float`: covarianza tra `x` e `y`.

**Eccezioni**

`ValueError` se `len(x) ≠ len(y)`.

---

### Implementazione

```
def covariance(x, y, w=None):
    x = np.asarray(x, dtype=float)
    y = np.asarray(y, dtype=float)
    if len(x) ≠ len(y):
        raise ValueError("x and y must have the same length")
    return weighted_mean(x * y, w) - weighted_mean(x, w) * weighted_mean(y, w)
```

Dopo aver verificato che `x` e `y` abbiano la stessa lunghezza, la funzione sfrutta la stessa identità algebrica usata per la varianza, applicata al prodotto $x_i y_i$.

### Significato matematico

La covarianza sfrutta l'identità:

$$\mathrm{Cov}(x, y) = \overline{xy} - \bar{x}\,\bar{y}$$

I tre termini nel codice corrispondono a:

$$\texttt{weighted_mean(x * y, w)} = \overline{xy}_w = \frac{\sum_i w_i\, x_i\, y_i}{\sum_i w_i}$$

$$\texttt{weighted_mean(x, w) * weighted_mean(y, w)} = \bar{x}_w\, \bar{y}_w = \frac{\sum_i w_i\, x_i}{\sum_i w_i} \cdot \frac{\sum_i w_i\, y_i}{\sum_i w_i}$$

Il risultato è equivalente alla forma pesata classica:

$$\mathrm{Cov}_w(x, y) = \frac{\sum_i w_i (x_i - \bar{x}_w)(y_i - \bar{y}_w)}{\sum_i w_i}$$

Una covarianza positiva indica che $x$ e $y$ tendono a crescere insieme; negativa, che crescono in direzioni opposte; zero, che sono linearmente indipendenti.

### Esempio d'uso

```
from stats_utils import covariance

x = [1.0, 2.0, 3.0, 4.0]
y = [2.1, 3.9, 6.2, 7.8]   # y cresce con x -> covarianza positiva
print(covariance(x, y))     # ≈ 2.425
x2 = [1.0, 2.0, 3.0]
y2 = [1.0, 2.0, 3.0]
print(covariance(x2, y2))   # = variance(x2) ≈ 0.667
```

## 2.5 standard_deviation(x, w=None)

**Scopo**
Calcolare la deviazione standard di un array, con possibilità di pesare ogni dato.

**Parametri**

- `x` array-like → convertito in `float64`
- `w` array-like o `None` (default: `None`)

**Vincoli**
`x` non deve essere vuoto ($N \geq 1$). Se `w` $\neq$ `None`, i pesi devono essere pre-calcolati come $w_i = 1/\sigma_i^2$ e $\sum_i w_i > 0$.

**Restituisce**
`float`: deviazione standard di `x`.

**Eccezioni**
`ValueError` se `x` è vuoto ($N = 0$).

### Implementazione

```
def standard_deviation(x, w=None):
    x = np.asarray(x, dtype=float)
    if len(x) == 0:
        raise ValueError("x must contain at least one value")
    return np.sqrt(variance(x, w))
```

Prima di procedere, la funzione controlla che `x` non sia vuoto, poi restituisce la radice quadrata della varianza calcolata da `variance`.

### Significato matematico

La deviazione standard è la radice quadrata della varianza:

$$\sigma(x) = \sqrt{\mathrm{Var}(x)}$$

Con i pesi la formula completa è:

$$\sigma_w(x) = \sqrt{\frac{\sum_i w_i x_i^2}{\sum_i w_i} - \left(\frac{\sum_i w_i x_i}{\sum_i w_i}\right)^2}$$

Senza pesi ($w_i = 1$ per tutti) si riduce alla forma classica:

$$\sigma(x) = \sqrt{\frac{1}{N}\sum_i (x_i - \overline{x})^2}$$

La deviazione standard ha le stesse unità di misura dei dati, a differenza della varianza che è espressa in unità al quadrato: è quindi la grandezza tipicamente riportata come incertezza statistica di una serie di misure.

### Esempio d'uso

```
from stats_utils import standard_deviation

x = [2.0, 4.0, 4.0, 4.0, 5.0, 5.0, 7.0, 9.0]
print(standard_deviation(x))   # 2.0
# media = 5, varianza = 4 -> sqrt(4) = 2
```

Capitolo 3

# plot_utils.py

Il file *plot_utils.py* raccoglie le funzioni di visualizzazione utilizzate per rappresentare graficamente i dati sperimentali. Per garantire accessibilità, tutti i colori usati nei grafici sono scelti da una palette *colorblind-safe* e definiti come costanti globali:

- `C_BAR = "#4878CF"`: blu, usato per le barre dell'istogramma;
- `C_MEAN = "#D65F5F"`: rosso, usato per la linea della media;
- `C_BAND_A = "#4878CF"`: blu, usato per la banda $\sigma_A$;
- `C_BAND_B = "#EE854A"`: arancione, usato per la banda $\sigma_{\text{tot}}$.

Le funzioni disponibili sono:

- `histogram(x, ...)`: genera un istogramma dei dati, con linea della media e banda di deviazione standard opzionali.

la documentazione per questa funzione è divisa in sottosezioni per spiegare al meglio ogni sua componente

## 3.1 histogram(x, bins, label, xlabel, ylabel, title, figsize, save_path, show_mean, show_std, dpi, ax)

**Parametri**

| | |
|---|---|
| **x** | array-like → convertito in `float64` |
| **bins** | intero o `"auto"` (default `"auto"`) → numero di intervalli |
| **label** | stringa (default `"Dati"`) → testo nella legenda |
| **xlabel** | stringa (default `"Valore1"`) → etichetta asse $x$ |
| **ylabel** | stringa o `None` → etichetta asse $y$ (default: `"Conteggi"`) |
| **title** | stringa (default `"Istogramma"`) → titolo del grafico |
| **figsize** | tupla (default `(8, 5)`) → dimensioni in pollici |
| **save_path** | stringa o `None` → percorso di salvataggio (default: `None`) |
| **show_mean** | `bool` (default `True`) → mostra la linea della media |
| **show_std** | `bool` (default `True`) → mostra la banda $\pm\sigma$ |
| **dpi** | intero (default `300`) → risoluzione della figura |
| **ax** | `Axes` o `None` → asse su cui disegnare (default: `None`) |

**Vincoli**

Se `ax` ≠ `None`, la figura viene disegnata sull'asse passato senza crearne una nuova. Se `save_path` ≠ `None`, la figura viene salvata nel percorso specificato con risoluzione `dpi`.

**Restituisce**

`int`: `0` (implementazione in corso).

**Errori**

Nessuno.

### 3.1.1 Parametri e figura

La funzione **histogram(x, ...)** genera un istogramma dei dati contenuti in `x`. Se non viene passato un asse `ax`, crea una nuova figura con le dimensioni e la risoluzione specificate.

```
def histogram(
    x,
    bins="auto",
    label="Dati",
    xlabel="Valore1",
    ylabel=None,
    title="Istogramma",
    figsize=(8, 5),
    save_path=None,
    show_mean=True,
    show_std=True,
    dpi=300,
    ax=None,
):
    x = np.asarray(x, dtype=float)

    # --- Figura ---
    if ax is None:
        fig, ax = plt.subplots(figsize=figsize, dpi=dpi, constrained_layout=True)
    else:
        fig = ax.get_figure()
```

Il primo passo converte l'input in un array `float64`. La gestione dell'asse permette di integrare la funzione sia in figure autonome sia in griglie di sottografici: se `ax = None` viene creata una nuova figura, altrimenti si usa l'asse fornito e si recupera la figura padre con `ax.get_figure()`. Il parametro `ax` sta per *axes* in inglese e fa parte della libreria `matplotlib`.

### 3.1.2 Istogramma

```
# --- Istogramma ---
counts, bin_edges, patches = ax.hist(
    x,
    bins=bins,
    density=False,
    color="#4878CF",
    edgecolor="white",
    alpha=0.85,
    label="Dati",
)
```

**Parametri**

| | |
|---|---|
| **bins** | =bins → numero di intervalli (default: "auto", scelti da NumPy) |
| **density** | =False → asse $y$ a conteggi assoluti |
| **color** | ="#4878CF" → colore delle barre |
| **edgecolor** | ="white" → bordo bianco tra una barra e l'altra |
| **alpha** | =0.85 → leggera trasparenza per distinguere meglio le barre |
| **label** | =label → testo nella legenda (default: "Dati") |

La funzione `ax.hist()` disegna l'istogramma e restituisce tre cose:

1. **counts**: un array con il numero di misure cadute in ogni bin
2. **bin_edges**: i bordi degli intervalli, include sia il bordo destro che quello sinistro (ha un elemento in più di counts)
3. **patches**: oggetti grafici delle barre (modificabili successivamente)

### 3.1.3 Elementi dell'istogramma

I componenti principali dell'istogramma sono le statistiche:

```
# --- statistiche ---
mu = np.mean(x)
sigma = mlt.standard_deviation(x)
```

dove `mu` rappresenta il valore atteso $E[x]$ e sigma è la **deviazione standard** calcolata con la funzione presente in 2.5.

Successivamente abbiamo la linea della media:

```
# --- linea media ---
if show_mean:
    ax.axvline(
        mu,
        color=C_MEAN,
        linestyle="--",
        linewidth=1.2,
        label=rf"$\bar{{x}} = {mu:.3g}$",
    )
```

La funzione `ax.axvline()` disegna una linea verticale in corrispondenza di $x = \mu$, permettendo di visualizzare immediatamente la posizione centrale della distribuzione secondo la media aritmetica. Questa indicazione è utile per confrontare il centro dell'istogramma con la dispersione dei dati e con eventuali altre statistiche descrittive.

Un ulteriore elemento grafico utile è la banda di dispersione centrata sulla media, estesa di una deviazione standard a sinistra e a destra:

```
 # --- linee +- sigma ---
if show_std:
    ax.axvspan(
        mu - sigma,
        mu + sigma,
        color="#EE854A" ,
        alpha=0.15,
        label=rf"$\pm 1\sigma = {sigma:.3g}$",
    )
```

La funzione `ax.axvspan()` colora la regione compresa tra $\mu - \sigma$ e $\mu + \sigma$, mettendo in evidenza l'intervallo di ampiezza pari a una deviazione standard attorno al valore centrale. Questa banda fornisce una rappresentazione visiva immediata della dispersione dei dati intorno alla media.

Si impostano poi le etichette degli assi, il titolo del grafico e la legenda:

```
 # --- etichette ---
 ax.set_xlabel(xlabel)
 ax.set_ylabel(ylabel if ylabel is not None else "Conteggi")
 ax.set_title(title)

 ax.legend(fontsize=9, framealpha=0.9)
```

L'etichetta dell'asse $x$ è assegnata tramite il parametro `xlabel`, mentre per l'asse $y$ viene utilizzata l'etichetta fornita in `ylabel`; se questa non è specificata, viene usato il valore predefinito `"Conteggi"`. Il titolo del grafico è fissato da `title`. Infine, `ax.legend()` aggiunge una legenda che raccoglie e descrive gli elementi grafici opzionali, come la media e la banda di dispersione.

Infine, il grafico può essere salvato su file se viene specificato un percorso di output:

```
# --- salvataggio ---
 if save_path is not None:
     fig.savefig(save_path, dpi=dpi, bbox_inches="tight")
```

La chiamata a `fig.savefig()` esporta la figura nel percorso indicato da `save_path`. Il parametro `dpi` controlla la risoluzione dell'immagine salvata, mentre `bbox_inches="tight"` riduce i margini bianchi superflui attorno al grafico.

### 3.1.4 Esempio d'uso

Il codice seguente genera 200 campioni da una distribuzione normale con media $\mu = 0$ e deviazione standard $\sigma = 1$ tramite `numpy` e produce l'istogramma con le opzioni predefinite:

```
import numpy as np
from mech_lab_tools import plot_utils

data = np.random.normal(loc=0, scale=1, size=200)

fig, ax = plot_utils.histogram(
    data,
    bins=20,
    label="Campione simulato",
    xlabel="Valore",
    title="Distribuzione normale simulata",
)
```

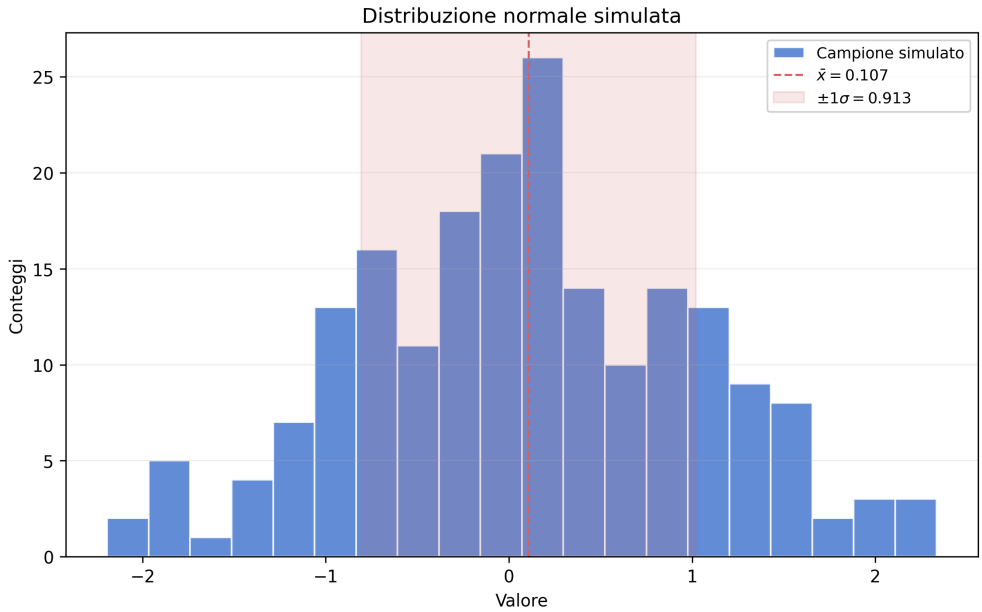


Figura 3.1: Istogramma di un campione di 200 valori estratti da una distribuzione normale $\mathcal{N}(0, 1)$.

Capitolo 4

# fit_utils.py

Il file *fit_utils.py* raccoglie le funzioni per il fit di dati sperimentali. Le funzioni supportano l'incertezza sulle misure tramite pesi $w_i = 1/\sigma_{y_i}^2$, che amplificano il contributo dei punti più precisi nella stima dei parametri.

Le funzioni disponibili sono:

- `lin_fit(x, y, sigma_y, ...)`: esegue un fit lineare pesato $y = a + bx$ con incertezze su $y$, restituisce i parametri stimati e le relative incertezze.

## 4.1 lin_fit(x, y, sigma_y, xlabel, ylabel, dpi, band, plot)

**Scopo**

Eseguire un fit lineare pesato $y = a + bx$ su un insieme di punti sperimentali con incertezze su $y$, stimando i parametri $a$ e $b$ e le loro incertezze.

**Parametri**

| | |
|---|---|
| **`x`** | array-like → convertito in `float64`; valori della variabile indipendente |
| **`y`** | array-like → convertito in `float64`; valori della variabile dipendente |
| **`sigma_y`** | array-like → convertito in `float64`; incertezze su $y$ (devono essere $> 0$) |
| **`xlabel`** | stringa (default `"x [xu]"`) → etichetta asse $x$ nel grafico |
| **`ylabel`** | stringa (default `"y [uy]"`) → etichetta asse $y$ nel grafico |
| **`dpi`** | intero (default `300`) → risoluzione della figura salvata |
| **`band`** | `bool` (default `True`) → se `True`, mostra la banda di confidenza attorno alla retta |
| **`plot`** | `bool` (default `True`) → se `True`, genera e mostra il grafico del fit |

**Vincoli**

`x`, `y` e `sigma_y` devono avere la stessa lunghezza e contenere almeno 3 elementi ($n \geq 3$). Tutti i valori di `x`, `y` e `sigma_y` devono essere finiti. Tutti i valori di `sigma_y` devono essere strettamente positivi ($\sigma_{y_i} > 0$) affinché i pesi $w_i = 1/\sigma_{y_i}^2$ siano finiti. Inoltre `x` deve contenere almeno due valori distinti, altrimenti la varianza pesata delle ascisse si annulla e il fit lineare non e' definito.

**Restituisce**

`dict` con i parametri del fit, le loro incertezze, i residui, la loro dispersione e la figura generata se `plot=True`.

**Eccezioni**

`ValueError` se `len(x) ≠ len(y)` o `len(x) ≠ len(sigma_y)`; `ValueError` se il numero di punti è inferiore a 3 ($n < 3$); `ValueError` se `x`, `y` o `sigma_y` contengono valori non finiti; `ValueError` se `sigma_y` contiene valori non strettamente positivi; `ValueError` se `x` non contiene almeno due valori distinti.

### 4.1.1 Pesi e stima dei parametri

La funzione **lin_fit(x, ...)** genera un fit lineare dei punti che vengono passati come input.

```
# --- pesi w_i = 1 / sigma_yi^2 ---
w = 1.0 / sigma_y**2

# --- stima dei parametri ---
var_x = mlt.variance(x, w)
cov_xy = mlt.covariance(x, y, w)

m = cov_xy / var_x
c  = mlt.weighted_mean(y,w) - m * mlt.weighted_mean(x, w)
```

questo primo stralcio di codice serve a calcolare il coefficiente angolare e l'intercetta della retta. Per farlo sfrutta le seguenti formule:

$$\hat{m} = \frac{\mathrm{Cov}_w(x, y)}{\mathrm{Var}_w(x)}, \qquad \hat{c} = \bar{y}_w - \hat{m}\,\bar{x}_w$$

dove $\bar{y}_w$ e $\bar{x}_w$ sono le medie pesate con $w_i = \frac{1}{\sigma^2_{y_i}}$ rispettivamente di $y$ e di $x$. Poniamo attenzione su come $w$ sia un `array-like`, in quanto esiste un peso per ogni $y$.

### 4.1.2 Incertezza sui parametri

Il codice seguente calcola le incertezze su i parametri calcolati precedentemente usando le formule della teoria

```
# --- incertezza sui parametri ---
sum_w = np.sum(w)

var_m = 1.0 / (var_x * sum_w)
var_c = mlt.weighted_mean(x**2, w) / (var_x * sum_w)

cov_mc = -mlt.weighted_mean(x, w) / (var_x * sum_w)

sigma_m = np.sqrt(var_m)
sigma_c = np.sqrt(var_c)
rho_mc = cov_mc / (sigma_m * sigma_c)
```

le formule sono le seguenti per la varianza:

$$\mathrm{Var}[\hat{m}] = \frac{1}{\mathrm{Var}[x]\sum_i \frac{1}{\sigma^2_{Y_i}}}, \qquad \mathrm{Var}[\hat{c}] = \frac{\overline{x^2}}{\mathrm{Var}[x]\sum_i \frac{1}{\sigma^2_{Y_i}}} = \overline{x^2}\cdot\mathrm{Var}[\hat{m}]$$

e per la covarianza:

$$\mathrm{Cov}[\hat{m}, \hat{c}] = -\frac{\overline{x}}{\mathrm{Var}[x]\sum_i \frac{1}{\sigma^2_{Y_i}}} = -\overline{x}\cdot\mathrm{Var}[\hat{m}]$$

le devizioni standard e il coeffiente di correlazione sono invece calcolati con le formule standard.

### 4.1.3 Residui e sigma dei residui

```
# --- residui e sigma dai residui (14.27) ---
r = y - m * x - c
sigma_r = np.sqrt(np.sum(r**2) / (n - 2))
```

In questo snippet di codice $r$ è il vettore dei residui, vale a dire la differenza tra il valore misurato $y_i$ e il valore previsto dalla retta:

$$r_i = y_i - (m \cdot x_i + c)$$

esprime quanto ogni punto si scosta dalla retta di fit.

La deviazione standard dei residui è invece calcolata come:

$$\sigma_r = \sqrt{\frac{\sum_i (y_i - (m \cdot x_i + c))^2}{n-2}}$$

quindi come la radice della somma in quadratura dei residui fratto $n-2$. Questo perché il fit lineare ha consumato 2 gradi di libertà per stimare $m$ e $c$.

### 4.1.4 Grafico dati, retta e residui

Dopo aver stimato parametri e residui, la funzione può costruire anche una visualizzazione del fit. Questa parte del codice è eseguita solo se `plot=True` e organizza il risultato in due pannelli: sopra i dati con la retta stimata, sotto i residui rispetto alla retta.

#### Figura con subplots

```
# --- grafico dati + retta di fit ---
fig = None
if plot:
    import matplotlib.pyplot as plt

    # figura con subplots
    fig, (ax_fit, ax_res) = plt.subplots(
        2,
        1,
        figsize=(8, 6),
        dpi=dpi,
        gridspec_kw={"height_ratios": [3, 1]},
        sharex=True,
        constrained_layout=True,
    )
```

La variabile `fig` viene inizializzata a `None` così che la funzione possa restituire comunque qualcosa anche quando il grafico non viene creato. Il blocco viene poi eseguito solo se `plot` è vero. L'import locale di `matplotlib.pyplot` serve a caricare la libreria grafica solo quando necessario.

La chiamata `plt.subplots(2, 1, ...)` crea una figura con due assi disposti in colonna: il primo, `ax_fit`, ospita il fit vero e proprio; il secondo, `ax_res`, ospita i residui. Il parametro `figsize=(8, 6)` imposta larghezza e altezza della figura in pollici, mentre `dpi=dpi` usa il valore passato alla funzione per fissare la risoluzione. Il dizionario `gridspec_kw` raccoglie le opzioni di layout dei due pannelli, così da assegnare più spazio alla parte superiore. Infine, `sharex=True` fa sì che i due assi condividano la stessa scala orizzontale, scelta naturale perché sia il fit sia i residui sono funzioni della stessa variabile $x$; `constrained_layout=True` chiede invece a Matplotlib di ottimizzare automaticamente gli spazi fra assi, etichette e legenda.

#### Pannello superiore

```
# pannello superiore: dati con error bar + retta
ax_fit.errorbar(
    x,
    y,
    yerr=sigma_y,
    fmt="o",
    color=C_BAR,
    markersize=4,
    ecolor=C_BAR,
    elinewidth=1,
    capsize=3,
)

x_fit = np.linspace(x.min(), x.max(), 200)
y_fit = m * x_fit + c
ax_fit.plot(x_fit, y_fit, color=C_MEAN, linewidth=1.5, label="Fit")

ax_fit.set_ylabel(ylabel)
ax_fit.grid(True, axis="y", linestyle="-", linewidth=0.5, alpha=0.3, zorder=0)
ax_fit.legend(fontsize=9, framealpha=0.9)
```

La chiamata `ax_fit.errorbar(...)` disegna i punti sperimentali con le rispettive barre d'errore. I primi due argomenti, `x` e `y`, sono le coordinate dei dati; `yerr=sigma_y` specifica invece l'incertezza verticale associata a ciascun punto. Il parametro `fmt="o"` chiede di rappresentare ogni misura con un marcatore circolare, `color=C_BAR` ne imposta il colore principale, `markersize=4` ne controlla la dimensione, `ecolor=C_BAR` usa lo stesso colore per le barre di errore, il parametro dedicato alla larghezza della barra ne regola lo spessore grafico, e `capsize=3` aggiunge i piccoli terminali orizzontali alle estremità delle barre.

Dopo i dati viene costruita la retta di fit. L'istruzione `np.linspace(x.min(), x.max(), 200)` genera 200 ascisse equispaziate tra il minimo e il massimo dei dati sperimentali: non sono nuovi dati misurati, ma una griglia regolare utile per tracciare una linea liscia. Su questa griglia la funzione calcola `y_fit = m * x_fit + c`, cioè l'equazione della retta stimata con i parametri del fit. La chiamata `ax_fit.plot(...)` disegna quindi la curva corrispondente: `color=C_MEAN` sceglie il colore della retta, `linewidth=1.5` ne fissa lo spessore, e `label="Fit"` fornisce il testo che comparirà in legenda.

Le ultime istruzioni rifiniscono il pannello. `ax_fit.set_ylabel(ylabel)` usa l'argomento della funzione per etichettare l'asse verticale; `ax_fit.grid(...)` aggiunge una griglia leggera solo sull'asse $y$, utile per stimare a colpo d'occhio lo scostamento dei punti dalla retta; `ax_fit.legend(...)` mostra infine la legenda, con dimensione del font e trasparenza del riquadro controllate dai due argomenti passati.

### Banda

```
# banda +- sigma sulla retta
if band:
    x_bar = mlt.weighted_mean(x,w)
    sigma_y_fit = np.sqrt(
        1.0 / sum_w + (x_fit - x_bar)**2 / (var_x * sum_w)
    )
    ax_fit.fill_between(
        x_fit,
        y_fit - sigma_y_fit,
        y_fit + sigma_y_fit,
        color=C_BAND_B, alpha=0.20,
        label=r"$\pm 1 \sigma$ retta"
    )
```

Il blocco è condizionato da `if band:`: la banda viene quindi disegnata solo quando l'utente lo richiede esplicitamente. La quantità `x_bar = mlt.weighted_mean(x, w)` è la media pesata delle ascisse e rappresenta il punto centrale attorno a cui la stima della retta è in genere più stabile. La variabile `sigma_y_fit` calcola invece, per ciascun valore di `x_fit`, l'ampiezza della fascia attorno alla retta: il primo termine dipende dal numero complessivo di informazioni disponibili tramite $\sum_i w_i$, mentre il secondo aumenta quando ci si allontana dalla zona centrale dei dati.

La funzione `ax_fit.fill_between(...)` colora la regione compresa tra il bordo inferiore `y_fit - sigma_y_fit` e quello superiore `y_fit + sigma_y_fit`. Il primo argomento, `x_fit`, fornisce l'asse orizzontale comune ai due bordi; `color=C_BAND_B` sceglie il colore della fascia, `alpha=0.20` la rende semitrasparente, e `label=...` aggiunge una voce dedicata nella legenda. Non è necessario ripetere qui la logica di colori e label già discussa nel pannello superiore: gli argomenti hanno la stessa funzione grafica, ma applicata a una regione anziché a una linea o a punti con barre d'errore. Visivamente, questa scelta rende immediato il confronto tra retta stimata e incertezza associata. La banda funziona quindi come un contesto grafico di lettura, non come un elemento separato dal fit. In altre parole, guida l'occhio prima ancora della lettura numerica.

### Pannello inferiore

```
# pannello inferiorre: residui
ax_res.errorbar(
    x,
    r,
    yerr=sigma_y,
    fmt="o",
    color=C_BAR,
    markersize=4,
    ecolor=C_BAR,
    elinewidth=1,
    capsize=3,
)
ax_res.axhline(0, color=C_MEAN, linewidth=1, linestyle="--")
ax_res.set_xlabel(xlabel)
ax_res.set_ylabel("Residui")
```

La chiamata `ax_res.errorbar(...)` riusa la stessa logica già descritta per `ax_fit.errorbar`: cambia però la grandezza rappresentata sull'asse verticale, perché al posto dei valori `y` vengono usati i residui `r`. In questo modo il pannello inferiore non mostra più i dati originali, ma quanto ciascun punto si discosta dalla retta di fit.

L'istruzione `ax_res.axhline(0, ...)` aggiunge una linea orizzontale di riferimento in corrispondenza di $r = 0$. Essa rappresenta il caso ideale in cui dato e modello coincidono perfettamente, quindi aiuta a vedere subito se i residui oscillano in modo bilanciato attorno a zero oppure mostrano andamenti sistematici. Infine, `ax_res.set_xlabel(xlabel)` etichetta l'asse comune delle ascisse, mentre l'ultima istruzione assegna al pannello l'etichetta verticale `"Residui"`.

**Return della funzione**

```
return {
    "m": m,
    "c": c,
    "sigma_m": sigma_m,
    "sigma_c": sigma_c,
    "cov_mc": cov_mc,
    "rho_mc": rho_mc,
    "r": r,
    "sigma_r": sigma_r,
    "fig": fig,
}
```

La funzione restituisce un dizionario, cioè una struttura che associa a ogni chiave testuale una quantità calcolata durante il fit. Le chiavi `"m"` e `"c"` contengono rispettivamente coefficiente angolare e intercetta; `"sigma_m"` e `"sigma_c"` contengono le loro incertezze; `"cov_mc"` e `"rho_mc"` descrivono invece la dipendenza statistica fra i due parametri, rispettivamente tramite covarianza e coefficiente di correlazione.

Le ultime tre voci riguardano qualità del fit e parte grafica. La chiave `"r"` restituisce il vettore dei residui, `"sigma_r"` fornisce la loro deviazione standard stimata, mentre `"fig"` contiene l'oggetto figura creato da Matplotlib se `plot=True`; se il grafico non viene richiesto, questa chiave rimane associata a `None`. In questo modo la funzione può essere usata sia in modalità puramente numerica sia in modalità numerica + grafica senza cambiare interfaccia.